

Embedded insights

Nathan Warner



**Northern Illinois
University**

Computer Science
Northern Illinois University
United States

Contents

Concepts

- **Electronics**
 - Voltage, current, resistance, and power
 - Ohm's law
 - Digital high and low voltage levels
 - GPIO input versus output
 - Pull-up and pull-down resistors
 - LEDs and current-limiting resistors
 - Push buttons and switch bouncing
 - Capacitors and basic RC circuits
 - Analog versus digital signals
 - Reading schematics and pinout diagrams
 - Common ground and safe voltage limits
 - Basic use of a multimeter and logic analyzer
- **MCU Architecture:** Learn how a microcontroller differs from a desktop processor:
 - CPU core, flash, SRAM, and peripherals
 - Memory-mapped I/O
 - Registers and special-function registers
 - Cortex-M4 register set
 - Vector table
 - Main stack pointer
 - Reset sequence
 - Privileged and unprivileged execution
 - Exceptions and interrupts
 - NVIC interrupt controller
 - SysTick timer
 - Memory protection unit
 - Floating-point unit
 - Little-endian representation
 - Thumb and Thumb-2 instructions
- **The embedded build process**
 - Cross-compilation
 - Preprocessor, compiler, assembler, and linker stages
 - Object files and ELF files
 - Sections such as .text, .rodata, .data, and .bss
 - Symbols and relocation
 - Linker scripts
 - Startup assembly
 - Vector-table placement

- Copying .data from flash to SRAM
- Zero-initializing .bss
- Map files
- Converting ELF to BIN or Intel HEX
- Flashing versus debugging

- **Essential STM32 peripherals:**

1. GPIO
2. Clock system and RCC
3. External interrupts
4. General-purpose timers
5. PWM
6. UART
7. ADC
8. SPI
9. I²C
10. DMA
11. Watchdogs
12. RTC and low-power modes

For every peripheral, understand:

- Which clock enables it
- Which GPIO alternate function it uses
- Which registers configure it
- How status flags work
- How polling differs from interrupts
- How DMA can move data without continuous CPU involvement
- What errors and edge cases can occur

Board exercises include LED blinking with a timer, button interrupts, UART command input, potentiometer sampling through the ADC, PWM brightness control, and reading a sensor through I²C or SPI.

- **Interrupts and concurrency:**

- Interrupt service routines
- Interrupt latency
- Interrupt priorities
- Shared state between main code and an ISR
- Atomic operations and critical sections
- Race conditions
- Reentrancy
- Keeping ISRs short
- Deferred processing
- Ring buffers

- Producer-consumer design
- Why blocking inside an ISR is dangerous
- Why volatile alone does not solve concurrency

Board exercise: receive UART bytes in an interrupt, place them into a ring buffer, and process them from the main loop.

- **Time and real-time behavior:**

- Busy waiting versus hardware timers
- Blocking versus nonblocking code
- Deadlines, latency, jitter, and throughput
- Periodic and event-driven tasks
- Finite-state machines
- Cooperative scheduling
- Monotonic timestamps
- Timer overflow
- Input capture and output compare
- Debouncing without blocking delays

Board exercise: run LED control, button handling, UART processing, and sensor sampling concurrently without calling a long blocking delay.

- **Hardware communication protocols:**

- **UART** Baud rate, frames, parity, buffering, overruns
- **SPI** Controller/peripheral roles, chip select, CPOL, CPHA, full duplex
- **I²C** Addresses, START/STOP, ACK/NACK, pull-ups, clock stretching
- **CAN** Frames, arbitration, identifiers, error handling
- **USB** Endpoints, enumeration, descriptors; learn later

- **Drivers and firmware architecture:** Learn to separate:

- Application logic
- Hardware-independent modules
- Device drivers
- Board support code
- MCU peripheral access
- Vendor HAL or low-level libraries

Also understand:

- Hardware abstraction layers
- Dependency injection through function pointers
- State machines
- Event queues
- Error reporting
- Timeouts
- Configuration versus runtime state

- Interfaces suitable for host-based testing
- **RTOS concept:** Do not begin with an RTOS. First become comfortable with interrupts, timers, state machines, and a superloop. Then learn
 - Tasks and context switching
 - Preemptive scheduling
 - Priorities
 - Queues
 - Semaphores
 - Mutexes
 - Software timers
 - Event groups
 - Priority inversion
 - Deadlock
 - ISR-safe APIs
 - Per-task stack sizing
 - Heap-allocation strategies
 - Tick frequency and tickless operation
- **For employment, you should at least recognize::**
 - Watchdog design
 - Brownout and reset handling
 - Defensive timeouts
 - Fault recovery
 - Firmware versioning
 - Bootloaders and firmware updates
 - CRCs and data integrity
 - Power loss during flash writes
 - Static analysis
 - Coding standards such as MISRA C
 - Unit, integration, and hardware-in-the-loop testing
 - Resource limits
 - Low-power design
 - Security fundamentals
- **Progression:**

Stage	Board work	Main concepts
1	LED and button	GPIO, C bit operations, schematics
2	Build outside the IDE	GCC, ELF, linker script, startup code
3	Timer-driven LED	clocks, timers, interrupts, NVIC
4	UART command console	serial communication, ring buffers
5	ADC and PWM	analog input, sampling, timer output
6	I ² C/SPI	sensor protocols, datasheets, reusable drivers
7	DMA data acquisition	concurrency, buffering, throughput
8	Cooperative application	state machines, events, nonblocking design
9	FreeRTOS application	tasks, queues, synchronization
10	Portfolio project	architecture, testing, fault handling

- **For job**

- Write and debug C confidently.
- Explain startup, linking, flash, SRAM, and the vector table.
- Configure basic peripherals from their documentation.
- Write interrupt-driven, nonblocking firmware.
- Use UART, SPI, I²C, timers, ADC, PWM, and DMA.
- Diagnose faults using GDB and hardware tools.
- Explain races, critical sections, queues, and RTOS scheduling.
- Structure firmware into testable drivers and application modules.
- Present one substantial board project with documentation and tests.

1.1 Electronics

- **GPIO:** GPIO stands for General-Purpose Input/Output. GPIO allows software running on a microcontroller to interact with electrical signals through its physical pins.

A GPIO pin can usually operate as:

- A digital input
- A digital output
- A connection to an internal peripheral, such as UART, SPI, I²C, or a timer
- An analog input or output, when supported

Software reads and writes registers, the GPIO hardware translates those register operations into electrical behavior at a pin.

When configured as an output, the MCU drives the pin to one of two logic levels:

Software value	Logical state	Approximate voltage
0	Low	0 V
1	High	Supply voltage, often 3.3 V

Writing a 1 to the pin drives it high, causing current to flow through the resistor and LED.

GPIO pin – resistor – LED – ground

- **Active-high versus active-low:** The previous circuit is active-high because setting the GPIO high turns on the LED.

3.3 V – resistor – LED – GPIO pin

This is active-low. You must inspect the board schematic to determine which arrangement is used.

- **Push buttons:** A push button is simply a mechanical switch. It does not produce a voltage or send data by itself. It either:
 - Connects two electrical points when pressed, or
 - Disconnects two electrical points when released.

The core difference between a normally open (NO) and normally closed (NC) button is their default state when you are not touching them

- **NO:** The internal contacts are apart. The circuit is open. When pressed the contacts touch and close the circuit. Current flows to turn something on.
- **NC:** The internal contacts touch each other. The circuit is complete. When pressed the contacts pull apart and open the circuit. Power stops.

A **momentary circuit button** (usually called a momentary push-button switch) is an electrical switch that turns a circuit on or off only while you are actively pressing it.

The opposite of a momentary switch is a latching switch (also called a maintained switch).

The blue user button on your NUCLEO-F446RE is a momentary, normally-open switch:

- **GPIO as an input:** When configured as an input, the MCU does not intentionally drive the pin. Instead, it measures whether the external pin voltage represents a logical low or high.

For a 3.3 V MCU, the interpretation is approximately:

- **Near 0 V:** logic 0
- **Near 3.3 V:** logic 1

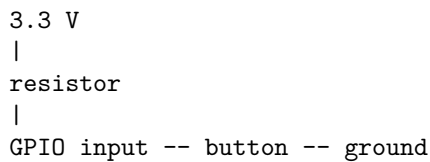
The exact voltage thresholds are given in the MCU datasheet.

- **Pull-up and pull-down resistors:** Pull-up and pull-down resistors are small components used in digital circuits to give a pin a defined default voltage level and stop it from "floating"

A floating pin is unconnected. It acts like an antenna, picking up stray electrical noise that causes random, unpredictable high or low readings

- **Pull up resistor:** A pull-up resistor connects the input pin directly to the positive voltage supply (V_{CC}). It holds the pin at a logical HIGH (1) when a switch or button is open.

When you close the switch, it connects the pin directly to ground. This overrides the weak pull-up resistor and pulls the reading to a logical LOW (0).



- **Pull down resistor:** A pull-down resistor connects the input pin directly to ground (0V). It holds the pin at a logical LOW (0) when a switch is open.

When you close the switch, it connects the pin to the voltage supply, overriding the resistor and pulling the pin to a logical HIGH (1).

Many MCUs contain configurable internal pull-up and pull-down resistors, so an external resistor is not always necessary.

- **The pin is not the same as the GPIO peripheral:** A physical MCU pin is connected to internal routing circuitry. That circuitry determines which internal component controls the pin.

A pin might be routed to:

- GPIO
- UART transmit or receive
- SPI clock or data
- I²C
- A timer PWM channel
- An ADC
- Another peripheral

Therefore, a pin marked PA5 does not necessarily have to act as an ordinary GPIO. It could be assigned an alternate function

- **Alternate function:** An alternate function on a microcontroller (MCU) is a special feature that lets a general-purpose input/output (GPIO) pin connect to an internal hardware peripheral instead of just acting as a basic digital input or output
- **STM32 GPIO naming:**
 - PA5 means port A, pin 5.
 - PB3 means port B, pin 3.
 - PC13 means port C, pin 13.

GPIO pins are organized into ports, usually containing up to 16 pins:

- **GPIOA:** PA0 through PA15
 - **GPIOB:** PB0 through PB15
 - **GPIOC:** PC0 through PC15
- **GPIO modes:** An STM32 GPIO pin commonly has four principal modes.

Mode	Purpose
Input	Read an external digital signal
Output	Drive the pin high or low
Alternate function	Connect the pin to UART, SPI, timers, etc.
Analog	Connect the pin to analog circuitry or reduce digital power consumption

Selecting the correct mode is essential. Writing an output value does not make a pin behave as an output if it is still configured as an input.

- **Output driver types:** When operating as an output, a GPIO pin can commonly use either push-pull or open-drain behavior.

A push-pull output can actively drive both voltage levels:

- Write 1 → pin driven high
- Write 0 → pin driven low

This is normally used for:

- LEDs
- Digital control signals
- SPI signals
- General output pins
- **Open-drain:** An open-drain output is a digital pin configuration that can pull a signal down to ground or disconnect completely, but it cannot actively drive a high voltage on its own

If any device pulls the line low, the line is low. The line becomes high only when every device releases it.

- **Output speed:** STM32 GPIO configuration includes an output speed setting. This does not normally mean how frequently software toggles the pin. It primarily controls how quickly the output voltage transitions between low and high.

A faster edge supports higher-frequency signals but can cause:

- More electromagnetic interference
- More power consumption
- More ringing and signal-integrity problems

For a slowly blinking LED, a low-speed setting is sufficient.

- **GPIO registers:** GPIO is controlled through memory-mapped registers. These registers occupy addresses in the MCU's memory map, but accessing them controls hardware rather than ordinary RAM.

For an STM32 GPIO port, important registers commonly include:

Register	Purpose
MODER	Select input, output, alternate-function, or analog mode
OTYPER	Select push-pull or open-drain output
OSPEEDR	Select output transition speed
PUPDR	Configure pull-up or pull-down resistors
IDR	Read current input states
ODR	Read or change output states
BSRR	Atomically set or reset output bits
AFRL, AFRH	Select alternate peripheral functions

Each pin is represented by one or more bits in these registers.

- **Configuring a pin as an output:** Suppose we want to configure PA5 as an output.

On STM32 devices, each pin has a two-bit field in MODER:

MODER field	Mode
00	Input
01	Output
10	Alternate function
11	Analog

Because PA5 is pin 5, its mode is controlled by bits 11 ($2(5) + 1$) and 10 ($2(5)$):

A simplified register-level configuration looks like:

```
0  GPIOA->MODER &= ~(3U << (5U * 2U)); // Clear both mode bits
1  GPIOA->MODER |= (1U << (5U * 2U)); // Select output mode
```

The first operation clears the two-bit field. The second sets it to 01.

You could modify the output data register:

```
0  GPIOA->ODR |= (1U << 5); // Set PA5 high
1  GPIOA->ODR &= ~(1U << 5); // Set PA5 low
```

However, STM32 generally provides the BSRR register for safer individual-bit changes:

```
0  GPIOA->BSRR = (1U << 5); // Set PA5 high
1  GPIOA->BSRR = (1U << (5 + 16)); // Set PA5 low
```

In BSRR:

- Bits 0–15 set corresponding pins.
- Bits 16–31 reset corresponding pins.

This avoids a read-modify-write sequence and makes the operation atomic with respect to interrupts.

BSRR means Bit Set/Reset Register. It is a 32-bit, write-only register divided into two halves. The important feature is that each operation requires only one write to the peripheral.

After the CPU writes to GPIOA->BSRR, the write travels through the STM32 bus system to the GPIOA peripheral. The BSRR does not store the value like an ordinary variable; GPIOA interprets it as a set/reset command.

- **Reading an input:** To test whether PC13 is high:

```

0  if (GPIOC->IDR & (1U << 13)) {
1      // PC13 is high
2  } else {
3      // PC13 is low
4  }

```

IDR contains one bit for each pin in the port. Reading IDR observes the pin's actual digital input state. Reading ODR observes the value stored in the output latch. These values can differ under certain electrical conditions.

- **RCC:** RCC stands for Reset and Clock Control, a dedicated hardware register block inside STM32 microcontrollers that manages system resets and distributes clock signals
- **STM32 bus architecture:** STM32 microcontrollers control internal system buses and attached hardware via memory-mapped control, clock, and configuration registers belonging to the Reset and Clock Control (RCC) subsystem and individual peripheral blocks

STM32 chips (such as the STM32F4/F7 series) use an AHB/APB bus matrix to connect the CPU and DMA masters to memory and peripherals

- **AHB (Advanced High-performance Bus):** Used for high-speed components like Flash, SRAM, and DMA. Divided into AHB1 and AHB2 for core/GPIO access
- **APB (Advanced Peripheral Bus):** Used for lower-speed peripherals. Divided into APB1 (timers, I2C, SPI2/3, UART2-5) and APB2 (high-speed SPI1, USART1, ADCs)

Peripherals on these buses are disabled by default after a reset to save power. You must enable their respective bus clocks using RCC registers before reading or writing to any peripheral register

Note: ENR stands for enable register

- **Enabling the GPIO peripheral clock:** On many MCUs, a GPIO peripheral does nothing until its clock is enabled. This reduces power consumption for unused hardware.

On the STM32F446RE, GPIO ports are connected to the AHB1 (advanced high performance bus 1) peripheral bus. GPIOA can be enabled with:

```

0  RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

```

The sequence for a simple output is therefore:

1. Enable the GPIO port's clock.
2. Configure the pin mode.
3. Configure its output type, speed, and pull resistors.
4. Write the desired output state.

```

0  // Enable GPIO port A clock
1  RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
2
3  // Clear port A pin 5
4  GPIOA->MODER &= ~(3U << (5U * 2U));
5
6  // Set port A pin 5 output
7  GPIOA->MODER |= (1U << (5U * 2U));
8
9  GPIOA->OTYPER &= ~(1U << 5); // Push-pull
10 GPIOA->PUPDR &= ~(3U << (5U * 2U));
11
12 // Set high
13 GPIOA->BSRR = (1U << 5);      // PA5 high

```

- **Electrical limitations:** A GPIO pin is a small digital interface, not a general-purpose power source.

Important restrictions include:

- Maximum voltage on the pin
- Maximum current per pin
- Maximum total current for a port or the MCU
- Whether the pin is 5 V tolerant
- Whether analog functionality is supported
- Whether the pin is reserved for debugging or another board function

- **GPIO and interrupts:** Instead of repeatedly checking an input:

```

0  while (1) {
1      if (GPIOC->IDR & (1U << 13)) {
2          // Handle input
3      }
4  }

```

the MCU can configure the pin to generate an interrupt when the signal changes.

Common trigger choices include:

- Rising edge: low to high
- Falling edge: high to low
- Both edges

- **Voltage labels:**

- V_{DD} represents the positive supply voltage for a circuit, particularly those using field-effect transistors (FETs) like CMOS chips. The abbreviation stands for Voltage at the Drain
- V_{SS} (Voltage at the Source): The negative supply or ground rail paired with V_{DD} in FET/CMOS circuits.

- V_{CC} / V_{EE} : The equivalents used in bipolar junction transistor (BJT) circuits. V_{CC} stands for Voltage at the Collector (positive), and V_{EE} stands for Voltage at the Emitter (negative/ground).
- **More on push-pull:** A GPIO pin configured as an output must electrically drive the external wire. The two common output-driver designs are push-pull and open-drain.

1.2 MCU Architecture

- **Intro:** A Microcontroller Unit (MCU) architecture integrates a central processing unit, memory, and input/output peripherals onto a single integrated circuit
 - **CPU (Central Processing Unit):** The brain that fetches, decodes, and executes program instructions. It includes the Arithmetic Logic Unit (ALU) for math and logic, and the Control Unit for sequencing operations
 - **Program memory (ROM / flash):** Non-volatile storage that holds your permanent application code so it is not lost when power turns off.
 - **Data Memory (RAM):** Volatile storage used by the CPU to temporarily save active variables and runtime data
 - **System Bus:** An internal highway of wires that lets the CPU talk to memory and hardware peripherals
 - **Peripherals:** Built-in hardware tools that let the chip interact with the physical world. Common examples include:
 - * **GPIO:** General-purpose input/output pins for turning things on or off.
 - * **ADC:** Analog-to-digital converters to read voltages from sensors.
 - * **Timers/Counters:** For measuring time, creating delays, or generating Pulse-Width Modulation (PWM) signals.
 - * **Communication Interfaces:** Protocols like UART, SPI, and I²C to talk to other chips
- **Von Neumann architecture:** Uses a single shared memory space and bus for both data and instructions. It is simple to design but creates a data bottleneck because the CPU cannot read an instruction and read/write data at the exact same time
- **Harvard architecture:** Uses physically separate memory blocks and separate buses for code and data. This lets the processor fetch an instruction while simultaneously accessing data, which speeds up real-time execution. Most modern MCUs use variations of this model
- **Execute-in-place (XiP):** instructions do not necessarily need to move from ROM to RAM before execution. Most modern microcontrollers execute code directly from flash memory (ROM) using a technique called Execute-in-Place (XiP).

In most microcontrollers (like ARM Cortex-M, AVR, or PIC), the CPU is wired to fetch instructions directly from the Flash/ROM bus.

Microcontrollers have very limited RAM (often only kilobytes) but larger Flash memory. Keeping the code in Flash saves precious RAM for data and variables.

Flash memory is generally slower than RAM. For ultra-high-speed operations, Flash access time can become a performance bottleneck.

If instructions are fetched from Flash while data is simultaneously read or written in RAM, that is a classic implementation of the Harvard architecture.

- **Flash memory:** Flash holds data without power. It normally contains:
 - The vector table
 - Machine instructions
 - Read-only constants
 - Initial values for initialized global variables

On the STM32F446, main flash begins at:

```
0 0x08 00 00 00
```

The processor normally executes instructions directly from flash. This is commonly called execute in place.

Flash is persistent but slower and more complicated to modify than SRAM. You cannot generally treat it as ordinary writable memory.

- **SRAM:** SRAM is volatile working memory. It loses its contents when power is removed.

It contains things such as:

- Global and static variables
- The stack
- The heap, if one is used
- Communication buffers
- Temporary computation results

The primary SRAM address range begins at:

```
0 0x20000000
```

Address range	Purpose
0x00000000...	Boot-memory alias
0x08000000–0x0807FFFF	512 KiB internal Flash
Mostly reserved/unmapped space	No physical memory responds here
Near 0x1FFF0000	Factory system memory and bootloader
0x20000000–0x2001FFFF	128 KiB SRAM

The address space is not one enormous physical memory device. Large address ranges are simply unused or reserved.

- **The boot-memory alias:** 0x00000000 is particularly important. At reset, the Cortex-M4 expects the vector table to appear there:

Address	Initial meaning
0x00000000	Initial Main Stack Pointer value
0x00000004	Address of the reset handler
0x00000008	Address of the NMI handler
0x0000000C	Address of the HardFault handler

- **Programs:** In ordinary embedded development, Flash contains one firmware image:

```
Flash
|-- Vector table
|-- Program instructions
|-- Read-only constants
|-- Initial values for global variables
```

That firmware might contain many modules, functions, tasks, or an RTOS, but collectively it is normally considered one program.

You can store multiple independent firmware images if you create a bootloader and partition Flash.

Region	Example size
Bootloader	32 KiB
Application A	200 KiB
Application B	200 KiB
Configuration/metadata	Remaining space

The bootloader would select an application, set the application's vector-table location, load its initial stack pointer, and jump to its reset handler.

Without a custom bootloader: normally one firmware program

- **Address space:** On the STM32F446RE, the address space is primarily a physical address space, not a virtual address space.

When the Cortex-M4 executes:

```
o LDR r0, [r1]
```

and r1 contains 0x20000100, that address is placed onto the MCU's internal bus. The bus interconnect routes it directly to the SRAM hardware.

Memory on the MCU is usually not virtual

- **Startup chain:**
 1. Reset happens
 2. boot mode selects memory
 3. CPU reads the vector table
 4. CPU enters the reset handler
 5. startup code calls main()
- **Resets:** A reset returns the processor and much of the microcontroller hardware to a known initial state. It does not erase the program stored in Flash.

A reset may be caused by:

- Power being applied
- The NRST reset pin being asserted
- A watchdog timeout
- Brownout or insufficient supply voltage
- Software requesting a reset
- A debugger pressing “reset”

Different reset sources affect slightly different parts of the MCU, but they generally restart program execution using the same boot process.

After reset:

- The CPU registers return to defined reset states.
 - Most peripherals return to their reset states.
 - The CPU uses the Main Stack Pointer, or MSP.
 - The CPU begins in privileged Thread mode.
 - Flash retains its contents.
 - Ordinary SRAM contents should be considered undefined, even if some electrical values happen to remain.
- **Bootting:** Booting is the process of deciding which code the processor should begin executing and then preparing that code to run.

For a normal embedded application, the process is:

1. Reset
2. Select boot memory
3. Read initial stack pointer
4. Read reset-handler address
5. Execute Reset_Handler
6. Initialize RAM and hardware
7. Call main()

The STM32F446 can boot from different physical memory regions. Boot configuration pins and option settings determine which region appears at address 0x00000000.

The primary choices are:

Boot source	Purpose
Main Flash	Run your normal firmware
System memory	Run ST’s factory-programmed bootloader
SRAM	Run a program previously loaded into SRAM

The Nucleo board normally holds BOOT0 low, selecting the main Flash.

Your program physically begins at:

```
1 0x08000000
```

During normal Flash boot, the STM32 aliases this memory at:

```
1 0x00000000
```

Both addresses refer to the same beginning of Flash during this boot configuration. The CPU reads the initial startup information from 0x00000000, while your linker normally places it physically at 0x08000000.

- **System bootloader:** STM32 contains factory-programmed code called the system bootloader. It can be used to program Flash through supported communication interfaces without using the normal debugger.

This factory bootloader is separate from your application and is not erased when you program ordinary Flash.

An MCU system bootloader is a dedicated piece of firmware stored in a device's non-volatile memory that executes immediately upon power-up or reset to initialize hardware and manage application loading or updates

Configures essential low-level hardware like clocks, memory regions, and input/output ports before handing control over

Usually sits at a fixed address range completely distinct from the main user flash memory (for example, starting at 0x1FFF0000 on certain STM32 chips).

This memory is write-protected at the factory level and cannot be erased or modified by your software. It is always available as a fallback recovery mechanism.

You activate the ST system bootloader when you want to program or recover the MCU without running the firmware currently stored in main flash.

- **SRAM boot:** The CPU can also start from SRAM. This is primarily useful for debugging, testing, or specialized loaders.

Because SRAM is volatile, the program must already have been loaded into SRAM by something such as a debugger.

- **Vector table:** The vector table is an array of 32-bit values that tells the Cortex-M4:
 1. What the initial stack pointer should be.
 2. Where the reset handler is.
 3. Where every exception and interrupt handler is.

It is called a “vector” table because each entry directs, or vectors, execution to a handler. For a normal application, the vector table is placed at the beginning of Flash:

The vector table and your program at 0x0800.. do not clash because the vector table is part of your program's flash image. When we say the program "starts at 0x08000000," that does not mean the first machine instruction is stored there.

Address	Contents
0x08000000	Initial Main Stack Pointer value
0x08000004	Address of Reset_Handler
0x08000008	Address of NMI_Handler
0x0800000C	Address of HardFault_Handler
...	Remaining exception and interrupt vectors
After vector table	Startup code, functions, constants, and other program data
Later address	main() machine code

Each independently bootable firmware program normally has its own vector table. The vector table belongs to the firmware image and tells the processor:

- Where its stack begins
- Where its reset code begins
- Where its exception and interrupt handlers are

- **Multiple programs in flash:** Suppose you have a bootloader and an application:

```

0  0x08000000 Bootloader vector table
1      Bootloader code
2
3  0x08020000 Application vector table
4      Application code

```

At reset, the processor uses the bootloader's vector table because it is at the boot address. When the bootloader wants to run the application, it reads the application's first two vector entries:

```

0  uint32_t app_msp    = *(uint32_t *)0x08020000;
1  uint32_t app_reset = *(uint32_t *)0x08020004;

```

It then approximately performs:

```

0  SCB->VTOR = 0x08020000; // Use application's vector table
1  __set_MSP(app_msp);     // Use application's stack
2  jump_to(app_reset);     // Run application's Reset_Handler

```

Therefore, multiple firmware images can coexist, but only one vector table is active for interrupt dispatch at a time.

- **The Main Stack Pointer:** The Main Stack Pointer, or MSP, is a CPU register containing the current top of the main stack.

On Cortex-M processors, register R13 serves as the stack pointer. There are actually two possible stack pointers:

- **MSP:** Main Stack Pointer
- **PSP:** Process Stack Pointer

The MSP is normally used:

- Immediately after reset
- During startup
- By exception and interrupt handlers
- Often by simple bare-metal applications for everything

The PSP is optional and is commonly used by an operating system to give application threads separate stacks.

The first vector-table entry contains the initial value that the processor loads into the MSP:

0	Address	Value	Meaning
1	0x08000000	0x20020000	Initial MSP
2	0x08000004	0x08000151	Reset_Handler address

During reset, the processor effectively performs:

```
0  MSP <- memory[0x00000000]
1  PC  <- memory[0x00000004]
```

When booting from main flash, those locations correspond to:

```
0  MSP <- memory[0x08000000]
1  PC  <- memory[0x08000004]
```

- 0x08000000 is where the initial MSP value is stored in flash.
- 0x20020000 is the SRAM address loaded into the MSP register.

Note: Stacks on Cortex-M normally grow toward lower addresses.

- **Peripherals:** A peripheral is a hardware unit that performs a specialized task.
 - **GPIO:** reads and controls pins
 - **USART/UART:** serial communication
 - **Timers:** timing, counting, PWM and input capture

- **ADC:** converts analog voltage into a number
- **DMA:** moves data without making the CPU copy every element
- **RCC:** controls clocks and resets
- **EXTI:** detects external pin events

A peripheral normally contains configuration registers, status registers and data registers.

- **Memory-mapped I/O:** The Cortex-M4 communicates with peripherals using ordinary memory addresses.

```
0  uint32_t value = *(volatile uint32_t *)0x40020010;
```

The C expression appears to read memory, but address 0x40020010 belongs to a GPIO peripheral. The access therefore reads a hardware register rather than SRAM.

When the CPU tries to access an address assigned to a peripheral, the hardware bus or memory management unit routes the request directly to the device's physical registers rather than to system RAM

A peripheral register can change for reasons outside the current C code. The compiler must perform each requested access rather than caching or eliminating it. Therefore register pointers are normally declared volatile:

```
0  volatile uint32_t *status =
1      (volatile uint32_t *)0x40020010;
```

- **Registers and special-function registers:** The word “register” has two related meanings in embedded programming.
 - **CPU registers:** These are storage locations directly inside the Cortex-M4 core:
 - * R0, R1, R2, ... R15
 - * xPSR
 - * CONTROL
 - * PRIMASK
 - * BASEPRI
 - * FAULTMASK

CPU registers are operated on directly by machine instructions.

- **Peripheral registers:** These are memory-mapped hardware control locations:
 - * GPIOA_MODER
 - * GPIOA_IDR
 - * GPIOA_ODR
 - * RCC_AHB1ENR
 - * TIM2_CR1
 - * USART2_DR

They are sometimes called special-function registers, or SFRs.

- **Common register types**

Type	Behavior
Read/write	Software can read and modify it
Read-only	Reports hardware status
Write-only	Writing commands the hardware
Write-one-to-clear	Writing 1 clears a status flag
Set/reset register	Different bits atomically set or clear outputs

This is why blindly applying code such as `reg |= mask` can be incorrect. The register's documented semantics must be checked first.

- **Cortex-M4 register set**

Register	Conventional role
R0–R3	Arguments, return values and temporary values
R4–R11	General-purpose, normally preserved by called functions
R12	Temporary scratch register
R13	Stack pointer
R14	Link register
R15	Program counter

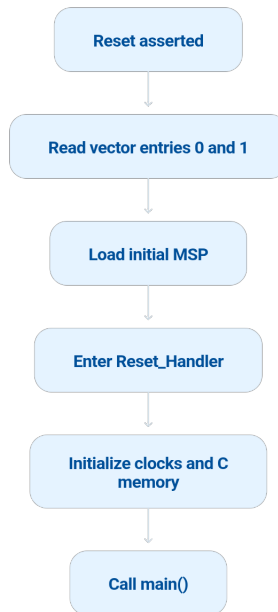
R14, is a dedicated core register in the ARM Cortex-M4 processor that holds the return address for function calls, subroutines, and exception handlers.

The calling convention, rather than the hardware alone, gives many of these registers their software roles.

- **Program status registers:** Several status components are collectively accessed as xPSR:
 - **APSR:** arithmetic flags
 - **IPSR:** current exception number
 - **EPSR:** execution-state information

Flag	Meaning
N	Negative result
Z	Zero result
C	Carry or unsigned condition
V	Signed overflow
Q	Saturation occurred

- **Reset sequence:** Reset is the bridge between powered hardware and your C main() function.



More precisely:

1. The processor enters reset state.
 2. The vector table is made visible at the boot address.
 3. The processor reads vector entry 0 and loads MSP.
 4. It reads vector entry 1 and loads the reset-handler address into PC.
 5. Execution begins in Reset_Handler.
 6. Startup code copies initialized data from flash into SRAM.
 7. Startup code clears the .bss section.
 8. System initialization may configure clocks and the FPU.
 9. C++ constructors are run when applicable.
 10. main() is called.
- **Why .data must be copied:** Consider

```
0  int counter = 5;
```

counter must be writable, so its runtime location is in SRAM. But SRAM loses its contents when powered off. Therefore the initial value 5 is stored in flash and copied into the SRAM location during startup.

- **Why .bss must be cleared:** Consider

```
0  static int errors;
```

C requires errors to begin as zero. Instead of storing thousands of zero bytes in flash, the linker places it in .bss, and startup code clears that memory range.

- **Privileged and unprivileged execution:** Cortex-M4 execution has two modes:
 - **Thread mode:** ordinary program or task execution
 - **Handler mode:** exception and interrupt handlers

Thread mode may be privileged or unprivileged. Handler mode is always privileged

Privileged software can perform operations such as:

- Configure the MPU
- Change interrupt masks
- Access restricted memory regions
- Modify important system-control registers

Unprivileged software is restricted.

The CONTROL.nPRIV bit controls Thread-mode privilege:

```
0  0 = privileged
1  1 = unprivileged
```

Privileged Thread mode can drop into unprivileged mode directly. Unprivileged Thread mode generally cannot restore its own privilege. It must request a privileged exception handler, commonly through SVC.

- **Exceptions and interrupts:** An exception is any event that causes the processor to suspend normal instruction execution and run a handler.

An interrupt is generally an asynchronous, hardware-generated kind of exception

- **Synchronous exceptions:** These result from the instruction being executed:
 - Undefined instruction
 - Invalid memory access
 - Divide-by-zero trap, when enabled
 - Supervisor call through SVC
- **Asynchronous interrupts:** These arrive from hardware independently of the current instruction:
 - Timer expiration
 - UART byte received
 - GPIO input edge
 - DMA transfer completed
 - ADC conversion completed
- **Exception entry:** When an exception is accepted, the processor automatically saves a basic context frame containing:

0	R0
1	R1
2	R2
3	R3
4	R12
5	LR
6	PC
7	xPSR

It then:

1. Selects MSP for Handler mode.
2. Obtains the handler address from the vector table.
3. Updates the current exception state.
4. Begins executing the handler.

On exception return, the hardware restores the saved registers and resumes the interrupted code.

- **Priorities and nesting:** A higher-priority interrupt can preempt a lower-priority handler. On Cortex-M, a lower numerical priority value represents a higher logical priority.

0	priority 0 = highest
1	priority 3 = higher than priority 8
2	priority 15 = lower

The STM32F446 implements a finite number of priority bits, so not every bit in an 8-bit priority field is meaningful.

- **CMSIS:** CMSIS (Cortex Microcontroller Software Interface Standard) is a vendor-independent hardware abstraction layer developed by Arm that provides consistent software interfaces for Arm Cortex-M processor-based microcontrollers like the STM32.

CMSIS assigns negative interrupt numbers to core exceptions and nonnegative numbers to device-specific interrupts

- **Interrupt Service Routine (ISR):** Another name for interrupt handler
- **NVIC interrupt controller:** NVIC means Nested Vectored Interrupt Controller. It is part of the Cortex-M core-peripheral architecture, not an ordinary STM32 timer or GPIO peripheral.

The NVIC (Nested Vectored Interrupt Controller) is a dedicated hardware block inside ARM Cortex-M based STM32 microcontrollers that manages, prioritizes, and processes all asynchronous system exceptions and hardware interrupts

- **Interrupt Prioritization:** Decides which task gets attention first; lower numerical values mean higher urgency (Priority 0 is the highest).
- **Preemption and Nesting:** Allows higher-priority interrupts to interrupt (nest inside) lower-priority active interrupt service routines (ISRs).

- **Vector Table Lookup:** Directly points the CPU to the exact memory address of the correct Interrupt Service Routine (ISR) when a signal triggers.
- **Tail-Chaining:** Optimizes back-to-back interrupts by skipping stack pop/push operations to reduce latency.

The NVIC records whether each external interrupt is:

- Enabled
 - Pending
 - Active
 - Assigned a particular priority
- **TIM2:** STM32 TIM2 is a heavy-duty, general-purpose 32-bit timer found on many STM32 microcontrollers
 - **Peripheral versus NVIC responsibilities:** Suppose TIM2 reaches its programmed count:
 1. TIM2 sets its own update flag.
 2. If the timer's update interrupt is enabled, it signals the NVIC.
 3. The NVIC marks TIM2's interrupt pending.
 4. The CPU compares its priority with the current execution priority.
 5. The CPU enters *TIM2_IRQHandler*.
 5. Software clears the TIM2 update flag.
 6. The processor returns from the exception.

There are therefore usually two enable controls:

Peripheral interrupt enable + NVIC interrupt enable

Both must be configured.

Similarly, clearing the NVIC pending bit alone may not solve the problem. If the peripheral's source flag remains set, it can request the interrupt again

- **Interrupt enable:** An interrupt enable is a switch or control bit in a microcontroller that tells the processor whether it is allowed to respond to a specific hardware event or a global signal
- **SysTick timer:** SysTick is a simple timer integrated with the Cortex-M core. It is commonly used for:
 - A periodic operating-system tick
 - Millisecond timekeeping
 - Software delays
 - Scheduling timeouts

Its main registers are:

Register	Purpose
SYST_CSR / CTRL	Enable, clock selection, interrupt enable and status
SYST_RVR / LOAD	Reload value
SYST_CVR / VAL	Current counter value
SYST_CALIB	Implementation-specific calibration information

SysTick is convenient, but it is not a full replacement for STM32 general-purpose timers.

SysTick_Config() sets the reload value, clears the current count, enables the interrupt and starts the counter.

Its connection to interrupts is that it can automatically generate an interrupt whenever its countdown reaches zero.

```

0  volatile uint32_t milliseconds;
1
2  void SysTick_Handler(void)
3  {
4      ++milliseconds;
5  }
6
7  int main(void)
8  {
9      SysTick_Config(SystemCoreClock / 1000u);
10
11     while (1) {
12         // milliseconds is updated once per millisecond
13     }
14 }
```

1. SysTick counts downward while the processor executes main().
2. The counter reaches zero.
3. SysTick raises an interrupt request.
4. The processor temporarily pauses main().
5. The processor executes SysTick_Handler().
6. The handler increments milliseconds.
7. The processor returns to the interrupted location in main()

- **Memory Protection Unit:** The MPU divides the physical address space into regions and assigns access rules.

A region can specify:

- Read/write permissions
- Privileged versus unprivileged access
- Executable or execute-never
- Memory type and caching-related attributes
- Region size and base address

Possible uses include:

- Making flash read-only
- Preventing instruction execution from SRAM
- Protecting kernel memory from application tasks
- Detecting stack overflows with a guard region
- Restricting a task's peripheral access

If software violates a configured MPU rule, the processor normally raises a MemManage fault.

- **Floating-point unit:** The Cortex-M4 used by the STM32F446 contains an FPUv4-SP single-precision FPU.

Single precision means normal hardware support for 32-bit IEEE-754 float operations:

It does not provide the same hardware support for 64-bit double arithmetic. Double-precision operations may require software routines and can be considerably slower.

The floating-point register file includes registers conventionally named:

o S0-S31

There is also an FPSCR status/control register.

- **Floating point and interrupts:** If an interrupt handler uses the FPU, floating-point state might also need to be saved. Cortex-M4 supports lazy floating-point context preservation, which postpones stacking that state until the handler actually executes a floating-point instruction.

This reduces interrupt overhead when the FPU is enabled but a particular handler does not use it.

- **Little-endian representation:** The STM32F446 uses little-endian byte ordering for ordinary data.
- **Thumb and Thumb-2 instructions:**

Context

The STMicroelectronics NUCLEO-F446RE is a highly popular, mid-range development board belonging to the STM32 Nucleo-64 ecosystem. It is designed around the powerful STM32F446RET6 microcontroller

Arm Cortex-M4 core processor

2.1 Documents

- **UM1724 — STM32 Nucleo-64 board user manual:** Board connectors, jumpers, power options, LEDs, buttons, clock configuration, ST-LINK, and Arduino/Morpho pin mappings
- **DS10693 — STM32F446xC/E datasheet:** Pinouts, alternate pin functions, voltage/current limits, clock limits, memory capacity, and electrical characteristics
- **RM0390 — STM32F446xx reference manual:** Peripheral behavior and registers: GPIO, RCC, timers, UART, SPI, I²C, ADC, DMA, interrupts, flash, USB, and more
- **PM0214 — STM32 Cortex-M4 programming manual:** CPU registers, instruction set, exception handling, NVIC, SysTick, MPU, FPU, and processor operating modes
- **MB1136 board schematic:** Exact electrical connections between the STM32, LEDs, buttons, oscillators, ST-LINK, and headers
- **ES0298 — STM32F446 device errata:** Known hardware limitations and their workarounds
- **UM1725 — STM32F4 HAL and LL driver manual:** STM32Cube HAL and low-layer driver APIs

2.2 Packages

- `arm-none-eabi-gcc` Compiles C/C++ for the Cortex-M4
- `arm-none-eabi-newlib` Embedded C standard library
- `arm-none-eabi-gdb` Debugs the program running on the board
- `openocd` Communicates with the board's integrated ST-LINK

2.3 STM32CubeMX

STM32CubeMX is a graphical hardware-configuration and code-generation program made by STMicroelectronics. It helps you configure an STM32 microcontroller without manually writing all the startup and peripheral-initialization code.

2.4 Generating a Project via CLI

Create a new text file named extension `.script`. Then,

```
1  # 1. Set the target microcontroller (replace with your exact part
    ↪  number)
2  config load STM32F466RETx
```

it is highly recommended to load the board configuration instead of just the MCU. Loading the board pre-configures system peripherals like the default USART pins for the built-in ST-LINK debugger, the user LEDs, and push buttons.

```
1  board load NUCLEO-F446RE
2
3  # 2. Setup project path and name
4  project name <project name>
5  project path <path>
6
7  # 3. Choose your IDE/Toolchain (e.g., STM32CubeIDE, Makefile,
    ↪  EWARM, MDK-ARM)
8  project toolchain Makefile
9
10 # Saves the generated .ioc (input output configuration)
11 config saveas <path>
12
13 # 4. Generate the code and project structure
14 project generate
15
16 # 5. Exit the CLI tool
17 exit
```

```
1  stm32cubemx -q *.script
```

2.5 The System

- **Cortex-M4 core:** Executes instructions and contains the CPU registers, exception mechanism, and SysTick timer.
- **STM32F446RE MCU:** Combines the Cortex-M4 core with Flash, SRAM, GPIO, timers, UARTs, ADCs, and other peripherals.
- **NUCLEO-F446RE board:** Connects the MCU to an onboard LED, button, ST-LINK debugger, USB connector, headers, and supporting circuitry.

2.6 Basics

•

2.7 Edit, Build, Flash, Debug cycle

Now that we have the project files, open the `.ioc` and verify

- PA5 is configured as GPIO_Output.
- PA5 controls the green onboard LED, labeled LD2.
- SYS → Debug is set to Serial Wire.
- System Core → GPIO shows PA5 as an output.
- A project toolchain is selected under Project Manager.

In `main.c`, have

```
0  while (1)
1  {
2      HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
3      HAL_Delay(500);
4
5      /* USER CODE END WHILE */
6
7      /* USER CODE BEGIN 3 */
8  }
```

Then, build the project with the generated makefile.

- make compiles and links.
- OpenOCD communicates with the onboard ST-LINK.
- arm-none-eabi-gdb provides source-level debugging.

Use `Build/*.elf` for flashing and debugging. It contains:

- Program instructions and data.
- Flash addresses.
- Function and variable names.
- Source-line debugging information.

Check the result with:

```
1  arm-none-eabi-size build/*.elf
```

	text	data	bss	dec
	↪ hex	filename		
2	5544	20	1644	7208
	↪ 1c28	intro.elf		

Connect the USB cable to the ST-LINK USB connector and run:

```
1 openocd -f board/st_nucleo_f4.cfg
```

Successful output should mention:

```
1 STLINK
2 stm32f4x.cpu
3 Listening on port 3333 for gdb connections
```

The supplied `st_nucleo_f4.cfg` configures the ST-LINK interface, SWD transport, and STM32F4 target.

To flash the program, run

```
1 openocd \
2     -f board/st_nucleo_f4.cfg \
3     -c "program build/*.elf verify reset exit"
```

This performs four operations:

1. Erases the necessary Flash sectors.
2. Programs the .elf
3. Verifies the written data.
4. Resets and starts the MCU.

Your LED program should begin running immediately.

Debugging uses two terminals. In the first terminal, start OpenOCD:

```
1 openocd -f board/st_nucleo_f4.cfg
```

Leave it running. OpenOCD exposes a GDB server on TCP port 3333. In the second terminal, start GDB:

```
1 arm-none-eabi-gdb -tui build/intro.elf
```

At the GDB prompt, enter:

```
1 target extended-remote localhost:3333
2 monitor reset halt
3 load
4 monitor reset halt
5 break main
6 continue
```

- **Target:** is a command used to specify the execution environment of the program you are debugging. It connects your local GDB session (the host) to a process, core file, or a separate machine (the target).
- **load:** The load command in GDB is primarily used during remote debugging to transfer and upload an executable file directly onto a remote target system (such as an embedded microcontroller or a remote board)
- **monitor reset halt:** In GDB, the command monitor reset halt is used during embedded firmware debugging to reset the target hardware and immediately halt the CPU at the very first instruction

Monitor tells GDB to pass the trailing command directly to the underlying hardware debug server (such as OpenOCD or pyOCD) rather than executing it within GDB itself

Reset halt instructs the debug server to assert the hardware reset line and stop the processor core immediately upon release. This prevents the MCU from executing any application code or bootloaders, keeping it in a clean, predictable state

The first resets and halts the MCU so OpenOCD can program Flash while the processor is not executing code.

The second resets the MCU after programming so the newly loaded firmware begins with a clean hardware state:

- Stack pointer is reloaded.
- Program counter returns to Reset_Handler.
- Global initialization runs again.
- Peripherals return to their reset state.

The first reset is not always strictly necessary because OpenOCD can normally halt the target automatically during load.

The complete connection is

GDB → OpenOCD → onboard ST-LINK → SWD → STM32F446RE

build/intro.elf contains:

- ARM machine instructions.
- The addresses where sections belong in Flash and SRAM.
- The interrupt vector table.
- Initialized global data.
- Function and variable names.
- Source-line debugging information.
- Section metadata such as .text, .data, .bss, and .isr_vector.

The MCU never parses the ELF file. OpenOCD reads the ELF metadata, extracts its loadable machine-code sections, and writes them into the correct Flash locations. After reset, the Cortex-M4 fetches instructions directly from Flash.

You can inspect the ELF structure with:

```
1 arm-none-eabi-readelf -h build/*.elf
2 arm-none-eabi-objdump -h build/*.elf
3 arm-none-eabi-nm build/*.elf
```

Once the firmware is flashed, the STM32 runs independently. GDB and OpenOCD do not need to remain connected. With

```
1 openocd \
2   -f board/st_nucleo_f4.cfg \
3   -c "program build/intro.elf verify reset exit"
```

The final reset exit means:

- Reset the STM32.
- Allow it to run.
- Exit OpenOCD.

After that, the program remains in Flash and automatically starts whenever you:

- Press the board's reset button.
- Disconnect and reconnect power.
- Turn the board off and back on.